

Project Description - CPEN333B

This project is a group assignment for CPEN333B. You are to work with your own group-mate, and one set of code files for each group is submitted to Canvas.

You can use the group tools available on Canvas to communicate with your groupmate (see the menu on the left).

Make sure you read the announcements regularly regarding the project.

The project is due **by 4:59 PM, the last day of classes**. Please plan ahead, no late submission will be accepted.

Academic honesty and standard:

As always, remind yourself of the [UBC academic honesty and standard \(https://vancouver.calendar.ubc.ca/campus-wide-policies-and-regulations/academic-honesty-and-standards\)](https://vancouver.calendar.ubc.ca/campus-wide-policies-and-regulations/academic-honesty-and-standards) and submit work that is yours or your own group's work only. Do not share/discuss your design thinking or any part of your code with anybody outside your own group. Using any outside snippet of code (that is neither from the course materials posted on Canvas, nor the Python's official online documentation) is not allowed. Any code snippet or code design from the the online documentation must be properly cited in your code.

Submission

Once you complete the implementations of the two parts of the project as described, submit your work to the associated submission dropbox for the project by the deadline. The code must include sufficient comment statements to document it well. No email or late submission is accepted, please do plan ahead accordingly. Coordinate among your group on who is to submit as only one set of files is to be submitted by the deadline (in the case of multiple submissions, we will only consider the last submission). Fully follow the project specification. Deliverables are:

- **Contribution statement file** (one .pdf file): clearly state the contribution each group mate has made (design, implementation, code review, ...). This should be reasonably detailed and must state the work each student has done on own.
- **Part1 code file** (one .py file): Submit one python3 file including the complete code for Part1 of the project. Make sure the file name includes the words `part1` in its name, e.g. `part1.py`.

- **Part1 additional documentation file** (one .pdf file): Submit a rather short documentation file (up to 5 pages) for the open-ended aspects of the project. This file does not replace the documentation you must have in the code as usual (readable code in the first place and useful comments statements and type hints), but it is meant to allow your group to provide your calculations, additional explanation, and your design consideration for any open-ended aspect of the project that you would need to document. The purpose is to further demonstrate and document your understanding of the dynamics of the game and its limitations.
 - The document must include:
 - Group number, student names, introduction
 - Design and implementation: clearly state different aspects of the implementation and design of open-ended aspects. For example, state how you implemented the pray capture (e.g. your calculation/geometry to explain under what conditions your program decides that the prey is captured), the movement, the pray placement, ... You must include graphic aids (illustrations, geometrical shapes, graphs, ...) to visually help with your explanation. Include any geometrical formulas or equations that you may have used. This section is meant to complement the code comments you have already must have included in the code.
 - Challenges: include any challenges that you faced, as well as, some incremental technical/implementation improvement(s) that you would consider for the future (explain only, but do not include in the submitted code).
- **Part2 code file** (one .py file): Submit one python3 file including the complete code for Part2 of the project. Make sure the file name includes the words `part2` in its name, e.g. `part2.py`.
- **Part2 additional documentation file** (one .pdf file): Submit a rather short documentation file (up to 5 pages), somewhat similar to the "*Part1 additional documentation file*", to document the design and implementation of this portion of your project. Clearly explain your design decisions and implementation.
- **iPeer survey** (linked on canvas under Project module): There will be an iPeer peer evaluation that you will fill out to rate each of your group-mates' level of contribution and effort in design, implementation and testing. This is an assessment point considered for a fair evaluation of individual grades. It is due is the same as the project files submission due. Completing the iPeer evaluation is required for each student and is considered for marking.

It is important to have regular communications with your group-mate on setting or re-assessing task assignments, integration and testing. It is natural to overestimate or underestimate, but you should adjust accordingly and timely.

Code review among group-mates is expected, that is, every group member's code used in the project must be reviewed by your other group-mate. This way, you not only help find bugs and improve coding, but every group-mate will learn about all aspects of the project. You cannot claim that that other group-mate coded this part, so you do not know about it.

=====

Part 1: Implementing a multithreaded game with graphic user interface

Write one python3 file the includes all the code for this part of the project. Make sure the file name includes the words part1 in its name.

We are to implement a simplified snake game in this project. Here are some characteristics of the program:

- Here is a video of the sample run of the game to show the basic intended behaviour:

<https://youtu.be/0qwd6bmTNXw> ⇨ <https://youtu.be/0qwd6bmTNXw>



<https://youtu.be/0qwd6bmTNXw>

- Note that the video uses "one" possible set of values for the widths of prey and the snake.
- The decision on when the prey is considered captured is left for your group. You must consider/document/implement a reasonable approach (e.g. when the prey is fully overlapped by the head of the moving snake (if the selected widths allow), ...)
- A new prey can be placed anywhere on canvas (considering the specified THRESHOLD) in its simplest form, but you should consider more reasonable limitations (e.g. not on the score text, not on the current position of the snake, ...)
- For a general, but longer and sped-up demo of a *generic* snake game see here:

[https://en.wikipedia.org/wiki/Snake_\(video_game_genre\)#/media/File:Snake_can_be_completed.gif](https://en.wikipedia.org/wiki/Snake_(video_game_genre)#/media/File:Snake_can_be_completed.gif) ⇨

[https://en.wikipedia.org/wiki/Snake_\(video_game_genre\)#/media/File:Snake_can_be_completed.gif](https://en.wikipedia.org/wiki/Snake_(video_game_genre)#/media/File:Snake_can_be_completed.gif)

- Python's `multithreading` module is used to implement multithreading in our program.
- Python's `Tkinter` module (<https://docs.python.org/3/library/tkinter.html> ⇨ <https://docs.python.org/3/library/tkinter.html>) is used to create the graphic user interface for our program. Under the hood Tkinter uses event-driven programming to implement concurrency. `Tkinter` is a popular and rather capable Python GUI module that comes as a part of Python's standard library (so no need to install anything additional). We have already used Tkinter in one of the labs.
- Python's `queue` module (<https://docs.python.org/3/library/queue.html> ⇨ <https://docs.python.org/3/library/queue.html>) is used as an excellent **thread-safe** multi-producer, multi-consumer queue (so we would not need any additional synchronization mechanism). Different tasks in our program are put in the FIFO queue and handled accordingly.

- The game is played by the four arrow keys of the keyboard bound to the corresponding directions, e.g. left arrow for the left direction, down arrow for the downward direction, ... This is done by the `Tkinter` 's provision of binding keys.
- The game is a variety of the snake game ([https://en.wikipedia.org/wiki/Snake_\(video_game_genre\)](https://en.wikipedia.org/wiki/Snake_(video_game_genre)))  ([https://en.wikipedia.org/wiki/Snake_\(video_game_genre\)](https://en.wikipedia.org/wiki/Snake_(video_game_genre))), which is a classic arcade game dating back to the early generations of computer games. The gamer controls a snake using the keyboard's arrow keys and the objective is to collect as many preys as possible (so the displayed game score is the number of collected preys). Each prey is stationary and is placed randomly on the game windows. As the snake feeds, it become longer (one additional block each time), progressively making it harder to control without losing.
- The game is over if the snake hits any of the walls or if it hits itself (technically biting itself).
- Many game related values are set using constants. For example, the speed of the game (the snake speed) is set to 0.15 (sec). Of course, you can adjust that during your debugging (if you need to slow it down or speed it up for your amusement). There are a few magic numbers left in the game that either you need to replace (when specified) or it should be rather very clear what they are.
- Fully follow the project specification. Write code, where it is specified. Unless it is asked, do not change any aspect of the project please.

Please read the docstring for each of the methods and comments for each portion of the program that is left for you to implement.

Also note that:

- The snake's current position and length is represented by the field `snakeCoordinates` of the `Game` class. It is a list of tuples. Each tuple is an (x, y) coordinate. In the GUI, the snake is represented by the `snakeIcon` which is a Tkinter canvas line with the width `SNAKE_ICON_WIDTH` based on the `snakeCoordinate` list.
- The new prey is created at the beginning of the game and each time the snake captures one. In the GUI, the prey is represented by the `preyIcon` which is a Tkinter canvas rectangle. The method `createNewPrey` randomly generates an x and a y, and then sets the prey `rectnagleCoordinates` as (x - 5, y - 5, x + 5, y + 5) which is added as a task to the queue. On the GUI, the prey is currently a 10 by 10 rectangle.
 - Currently, the magic number 10 (and its half, 5, and any other value related to it) is used for the prey width. You must modify the code so that instead of using the magic number 10, you use the constant `PREY_ICON_WIDTH` (add this to the code right after the line for `SNAKE_ICON_WIDTH`), so that we can easily change the width in one place.
 - Your program should work with any reasonable values used for the `SNAKE_ICON_WIDTH` or `SNAKE_ICON_WIDTH` (including the current values in the skeleton, obviously).
- Add any missing type hints for the method parameters and returns.

- You are allowed to use reasonably different values for the width (`WINDOW_WIDTH`) or/and length (`WINDOW_HEIGHT`) of the canvas widget, but justifications must be documented.
- The queue is a FIFO queue of all the pending tasks that each portion of the program created and added to the queue. Each item in the queue is a dictionary.
 - The keys in the dictionary is one of `"game_over"`, `"move"`, `"prey"`, or `"score"`.
 - The value depends on the key.
 - The value for the key `"game_over"` is a Boolean value (`True` or `False`)
 - The value for the key `"score"` is an integer representing the new score.
 - The value for the key `"prey"` is the new `rectangleCoordinates` of the form (x1, y1, x2, y2).
 - The value for the key `"move"` is the `snakeCoordinates` which is a list of tuples.
- The implementations of the *GUI* and the *Queue* classes, as well as the main thread is given to you. You are to complete the implementation of some of the methods in the *Game* class. Those two types and their operations must be used by the *Game* class whenever needed.
- Note that the `*` in an argument of the form `*var` in a function call is an *unpacking operator*. [From the documentation](https://docs.python.org/3/reference/expressions.html#expression-lists)  (<https://docs.python.org/3/reference/expressions.html#expression-lists>): "An asterisk `*` denotes iterable unpacking. Its operand must be an iterable. The iterable is expanded into a sequence of items, which are included in the new tuple, list, or set, at the site of the unpacking." Also in [PEP 448](https://peps.python.org/pep-0448/)  (<https://peps.python.org/pep-0448/>). (A note that `*` has different meaning for the method's formal parameter as opposed to a function argument.)

The amount of code that you will be writing is not much, but you need to understand the problem and figure out how to work with the provided classes and code to complete the project. This is a common situation when you join a company in real-life and need to continue on and complete a project based on present code and client requirements.

You are allowed to add inner functions to any of the methods but do not add any new type, or new methods. As usual, any new function must have its own docstrings and documentation. Do not modify the code skeleton, except for where required/specified.

Make sure that your code is readable and add comments whenever needed to explain your code.

Here is the code template:

```
# Group#:
# Student Names:

"""
```

```
This program implements a variety of the snake
game (https://en.wikipedia.org/wiki/Snake_(video_game_genre))
"""

import threading
import queue          #the thread-safe queue from Python standard library

from tkinter import Tk, Canvas, Button
import random, time

class Gui():
    """
    This class takes care of the game's graphic user interface (gui)
    creation and termination.
    """
    def __init__(self):
        """
        The initializer instantiates the main window and
        creates the starting icons for the snake and the prey,
        and displays the initial gamer score.
        """
        #some GUI constants
        scoreTextXLocation = 60
        scoreTextYLocation = 15
        textColour = "white"
        #instantiate and create gui
        self.root = Tk()
        self.canvas = Canvas(self.root, width = WINDOW_WIDTH,
                             height = WINDOW_HEIGHT, bg = BACKGROUND_COLOUR)
        self.canvas.pack()
        #create starting game icons for snake and the prey
        self.snakeIcon = self.canvas.create_line(
            (0, 0), (0, 0), fill=ICON_COLOUR, width=SNAKE_ICON_WIDTH)
        self.preyIcon = self.canvas.create_rectangle(
            0, 0, 0, 0, fill=ICON_COLOUR, outline=ICON_COLOUR)
        #display starting score of 0
        self.score = self.canvas.create_text(
            scoreTextXLocation, scoreTextYLocation, fill=textColour,
            text='Your Score: 0', font=("Helvetica", "11", "bold"))
        #binding the arrow keys to be able to control the snake
        for key in ("Left", "Right", "Up", "Down"):
            self.root.bind(f"<Key->{key}>", game.whenAnArrowKeyIsPressed)

    def gameOver(self):
        """
        This method is used at the end to display a
        game over button.
        """
        gameOverButton = Button(self.canvas, text="Game Over!",
                                height = 3, width = 10, font=("Helvetica", "14", "bold"),
                                command=self.root.destroy)
```

```
self.canvas.create_window(200, 100, anchor="nw", window=gameOverButton)
```

```
class QueueHandler():
    """
    This class implements the queue handler for the game.
    """
    def __init__(self):
        self.queue = gameQueue
        self.gui = gui
        self.queueHandler()

    def queueHandler(self):
        """
        This method handles the queue by constantly retrieving
        tasks from it and accordingly taking the corresponding
        action.
        A task could be: game_over, move, prey, score.
        Each item in the queue is a dictionary whose key is
        the task type (for example, "move") and its value is
        the corresponding task value.
        If the queue.empty exception happens, it schedules
        to call itself after a short delay.
        """
        try:
            while True:
                task = self.queue.get_nowait()
                if "game_over" in task:
                    gui.gameOver()
                elif "move" in task:
                    points = [x for point in task["move"] for x in point]
                    gui.canvas.coords(gui.snakeIcon, *points)
                elif "prey" in task:
                    gui.canvas.coords(gui.preyIcon, *task["prey"])
                elif "score" in task:
                    gui.canvas.itemconfigure(
                        gui.score, text=f"Your Score: {task['score']}")
                    self.queue.task_done()
            except queue.Empty:
                gui.root.after(100, self.queueHandler)

class Game():
    """
    This class implements most of the game functionalities.
    """
    def __init__(self):
        """
        This initializer sets the initial snake coordinate list, movement
        direction, and arranges for the first prey to be created.
        """
```

```
self.queue = gameQueue
self.score = 0
#starting length and location of the snake
#note that it is a list of tuples, each being an
# (x, y) tuple. Initially its size is 5 tuples.
self.snakeCoordinates = [(495, 55), (485, 55), (475, 55),
                        (465, 55), (455, 55)]
#initial direction of the snake
self.direction = "Left"
self.gameNotOver = True
self.createNewPrey()

def superloop(self) -> None:
    """
        This method implements a main loop
        of the game. It constantly generates "move"
        tasks to cause the constant movement of the snake.
        Use the SPEED constant to set how often the move tasks
        are generated.
    """
    SPEED = 0.15    #speed of snake updates (sec)
    while self.gameNotOver:
        #complete the method implementation below
        pass #remove this line from your implementation

def whenAnArrowKeyIsPressed(self, e) -> None:
    """
        This method is bound to the arrow keys
        and is called when one of those is clicked.
        It sets the movement direction based on
        the key that was pressed by the gamer.
        Use as is.
    """
    currentDirection = self.direction
    #ignore invalid keys
    if (currentDirection == "Left" and e.keysym == "Right" or
        currentDirection == "Right" and e.keysym == "Left" or
        currentDirection == "Up" and e.keysym == "Down" or
        currentDirection == "Down" and e.keysym == "Up"):
        return
    self.direction = e.keysym

def move(self) -> None:
    """
        This method implements what is needed to be done
        for the movement of the snake.
        It generates a new snake coordinate.
        If based on this new movement, the prey has been
        captured, it adds a task to the queue for the updated
        score and also creates a new prey.
        It also calls a corresponding method to check if
```

```

    the game should be over.
    The snake coordinates list (representing its length
    and position) should be correctly updated.
    """
    NewSnakeCoordinates = self.calculateNewCoordinates()
    #complete the method implementation below

def calculateNewCoordinates(self) -> tuple:
    """
        This method calculates and returns the new
        coordinates to be added to the snake
        coordinates list based on the movement
        direction and the current coordinate of
        head of the snake.
        It is used by the move() method.
    """
    lastX, lastY = self.snakeCoordinates[-1]
    #complete the method implementation below

def isGameOver(self, snakeCoordinates) -> None:
    """
        This method checks if the game is over by
        checking if now the snake has passed any wall
        or if it has bit itself.
        If that is the case, it updates the gameNotOver
        field and also adds a "game_over" task to the queue.
    """
    x, y = snakeCoordinates
    #complete the method implementation below

def createNewPrey(self) -> None:
    """
        This methods picks an x and a y randomly as the coordinate
        of the new prey and uses that to calculate the
        coordinates (x - 5, y - 5, x + 5, y + 5). [you need to replace 5 with a constant]
        It then adds a "prey" task to the queue with the calculated
        rectangle coordinates as its value. This is used by the
        queue handler to represent the new prey.
        To make playing the game easier, set the x and y to be THRESHOLD
        away from the walls.
    """
    THRESHOLD = 15    #sets how close prey can be to borders
    #complete the method implementation below

if __name__ == "__main__":
    #some constants for our GUI
    WINDOW_WIDTH = 500
    WINDOW_HEIGHT = 300

```

```
SNAKE_ICON_WIDTH = 15
#add the specified constant PREY_ICON_WIDTH here

BACKGROUND_COLOUR = "green"    #you may change this colour if you wish
ICON_COLOUR = "yellow"         #you may change this colour if you wish

gameQueue = queue.Queue()      #instantiate a queue object using python's queue class

game = Game()                  #instantiate the game object

gui = Gui()                    #instantiate the game user interface

QueueHandler()                 #instantiate the queue handler

#start a thread with the main loop of the game
threading.Thread(target = game.superloop, daemon=True).start()

#start the GUI's own event loop
gui.root.mainloop()
```

=====

Part 2: Producer-Consumer with Queue

We have looked at a few different implementations of the consumer-producer problem. In this part of the project, you are to implement another implementation based on the python's own queue type.

The queue (from the Queue module of the python standard library) is a thread-safe data structure (we have used it in part 1 of the project as well) and can be used as an alternative way to allow consumer and producer tasks to cooperate without the risk of race condition.

Notes, hints and requirements:

- Use the provided code skeleton.
- Use the queue as the buffer used to store and retrieve items. Use FIFO queuing. You can assume any reasonable type (float, string, mixed types, ... but stay consistent) for the items.
- Do not use any other synchronization primitive (lock, semaphore, ...), or buffering structure (circular buffer, ...).
- Do not include any other import other than the ones in the skeleton code.
- Include some reasonable randomness for when an item is produced or consumed.

- Use threading module types to create 4 producer threads and 5 consumer threads. Set to produce a reasonably manageable number of items (to simplify testing).
- Write the code with changeability in mind. All such values (number of threads, randomness, number of items, ...) should be easily changeable (e.g. do not use magic numbers).
- The program should gracefully end when all produced items are consumed. Consumer threads can be daemon threads (for simplicity), but as usual you need to explain and document your choice.
- Fully document your code as usual (that is include comments, docstrings for every method or type, type hints, ...).

Use the following code skeleton:

```
# Group #:
# Student names:

import threading
import queue
import time, random

def consumerWorker (queue):
    """target worker for a consumer thread"""
    # to implement

def producerWorker(queue):
    """target worker for a producer thread"""
    # to implement

if __name__ == "__main__":
    buffer = queue.Queue()
    # to implement
```